

```

// Movie Rating Sentinel - Daily Digest
// Parses DrunkenSlug RSS, filters by IMDb rating, sends email via SendGrid
//
// SETUP:
//   npm install node-fetch xml2js @sendgrid/mail dotenv
//
// SECRETS NEEDED:
//   RSS_FEED_URL, SENDGRID_API_KEY, EMAIL_FROM, EMAIL_TO
//
// CRON (6 AM EST = 11:00 UTC):
//   0 11 * * * cd /path/to/movie-digest && node movie-digest.js

"use strict";

require("dotenv").config();
const fetch = (...args) => import("node-fetch").then(({ default: f }) => f(...ar
const xml2js = require("xml2js");
const sgMail = require("@sendgrid/mail");

// -- Config -----

const CONFIG = {
  rssFeedUrl:    process.env.RSS_FEED_URL      || "",
  sendgridKey:   process.env.SENDGRID_API_KEY  || "",
  emailFrom:     process.env.EMAIL_FROM        || "",
  emailTo:       process.env.EMAIL_TO          || "",
  imdbThreshold: parseFloat(process.env.IMDB_THRESHOLD || "8.0"),
};

// -- Logging -----

const log = {
  info:    (m) => console.log(`[INFO]   ${new Date().toISOString()}  ${m}`),
  warn:    (m) => console.warn(`[WARN]   ${new Date().toISOString()}  ${m}`),
  error:   (m) => console.error(`[ERROR]  ${new Date().toISOString()}  ${m}`),
  success: (m) => console.log(`[OK]      ${new Date().toISOString()}  ${m}`),
};

// -- Step 1: Fetch RSS -----

async function fetchFeed(url) {
  log.info("Fetching RSS feed...");
  const res = await fetch(url, {
    headers: { "User-Agent": "MovieDigestBot/1.0" },

```

```

});
if (!res.ok) throw new Error(`RSS fetch failed: ${res.status} ${res.statusText}`);
const xml = await res.text();
const parsed = await xml2js.parseStringPromise(xml, { explicitArray: false });
const items = parsed?.rss?.channel?.item || [];
const arr = Array.isArray(items) ? items : [items];
log.info(`Feed returned ${arr.length} items`);
return arr;
}

// -- Step 2: Parse each item -----

function extractText(html, label) {
  const re = new RegExp(`- ${label}:\s*([<]+)<`, "i");
  const m = html.match(re);
  return m ? m[1].trim() : null;
}

function extractImdbId(item) {
  const attrs = item["nZEDb:attr"];
  if (attrs) {
    const arr = Array.isArray(attrs) ? attrs : [attrs];
    const imdbAttr = arr.find((a) => a.$?.name === "imdb");
    if (imdbAttr.$?.value) return imdbAttr.$?.value;
  }
  const desc = item.description?._ || item.description || "";
  const m = desc.match(/imdb\.com\/title\/tt(\d+)/i);
  return m ? m[1] : null;
}

function parseItem(item) {
  const rawTitle = item.title || "";
  const desc = item.description?._ || item.description || "";
  const imdbId = extractImdbId(item);

  const ratingStr = extractText(desc, "Rating");
  const imdb = ratingStr ? parseFloat(ratingStr) : null;

  // Strip resolution/codecs/release group from title
  // e.g. "Project Hail Mary 2026 2160p WebRip..." -> "Project Hail Mary"
  const titleMatch = rawTitle.match(/^(.+?)\s+(?:19|20)\d{2}\b/);
  const cleanTitle = titleMatch
    ? titleMatch[1].replace(/\.\/g, " ").trim()
    : rawTitle.replace(/\.\/g, " ").replace(/\s+(1080p|2160p|720p|BluRay|WEB|HDTV|

const year = extractText(desc, "Year");
const genre = extractText(desc, "Genre");

```

```

const director = extractText(desc, "Director");
const actors   = extractText(desc, "Actors");
const plot     = extractText(desc, "Plot");

const posterMatch = desc.match(/src="([^"]+covers\/movies\/[^\"]+)"/);
const poster = posterMatch ? posterMatch[1] : null;

return { rawTitle, cleanTitle, year, imdb, genre, director, actors, plot, poster }
}

// -- Step 3: Filter -----

function filterMovies(items) {
  const seen = new Set();
  const passed = [];

  for (const item of items) {
    const movie = parseItem(item);

    if (movie.imdb === null) {
      log.info(` skip (no rating): "${movie.cleanTitle}"`);
      continue;
    }

    const key = `${movie.cleanTitle.toLowerCase()}|${movie.year}`;
    if (seen.has(key)) continue;
    seen.add(key);

    if (movie.imdb >= CONFIG.imdbThreshold) {
      log.success(` PASS "${movie.cleanTitle}" (${movie.year}) - IMDb ${movie.imdb}`);
      passed.push(movie);
    } else {
      log.info(` skip "${movie.cleanTitle}" (${movie.year}) - IMDb ${movie.imdb}`);
    }
  }

  return passed;
}

// -- Step 4: Build Email -----

function buildHTML(movies) {
  const date = new Date().toLocaleDateString("en-US", {
    weekday: "long", year: "numeric", month: "long", day: "numeric",
  });

  const cards = movies.map((m) => `

```

```

<tr>
  <td style="padding:20px 0;border-bottom:1px solid #1e293b;">
    <table width="100%" cellpadding="0" cellspacing="0"><tr>
      ${m.poster ? `<td width="80" valign="top" style="padding-right:16px;">
        ` : ""}
      <td valign="top">
        <p style="margin:0 0 6px;font-size:18px;font-weight:600;color:#f1f5f9
          ${m.cleanTitle}
          <span style="font-size:13px;color:#475569;font-weight:400;">(${m.ye
        </p>
        <p style="margin:0 0 10px;">
          <span style="background:#14532d;color:#86efac;padding:3px 10px;bord
            IMDb ${m.imdb}/10
          </span>
        </p>
        ${m.genre ? `<p style="margin:0 0 3px;font-size:12px;color:#64748b
        ${m.director ? `<p style="margin:0 0 3px;font-size:12px;color:#64748b
        ${m.actors ? `<p style="margin:0 0 6px;font-size:12px;color:#475569
        ${m.plot ? `<p style="margin:0;font-size:13px;color:#94a3b8;line-
        ${m.imdbId ? `<p style="margin:8px 0 0;">
          <a href="https://www.imdb.com/title/tt${m.imdbId}/" style="color:#8
        </p>` : ""}
      </td>
    </tr></table>
  </td>
</tr>`).join("");

const emptyMsg = `
  <tr><td style="padding:32px 0;text-align:center;color:#475569;font-size:14px;
    No movies rated ${CONFIG.imdbThreshold}+ were found in today's feed.
  </td></tr>`;

return `<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,ini
<body style="margin:0;padding:0;background:#0a0a0f;font-family:Georgia,serif;">
<table width="100%" cellpadding="0" cellspacing="0" style="background:#0a0a0f;pad
  <tr><td align="center">
    <table width="600" cellpadding="0" cellspacing="0" style="max-width:600px;width

    <tr><td style="background:linear-gradient(135deg,#1a0a2e,#0f0a1e);border-radi
      <p style="margin:0 0 4px;font-size:11px;letter-spacing:0.3em;color:#7c3aed;
      <h1 style="margin:0 0 8px;font-size:28px;font-weight:400;color:#f1f5f9;lett
        Your Daily Top Films
      </h1>
      <p style="margin:0;font-size:13px;color:#475569;">${date}</p>

```

```

</td></tr>

<tr><td style="background:#0f172a;padding:14px 32px;border-bottom:1px solid #
  <p style="margin:0;font-size:13px;color:#64748b;font-family:monospace;">
    <strong style="color:#818cf8;">${movies.length} film${movies.length !== 1
      rated IMDb &gt;= ${CONFIG.imdbThreshold}/10 in today's feed
    </p>
  </td></tr>

<tr><td style="background:#0f172a;padding:0 32px;border-radius:0 0 12px 12px;
  <table width="100%" cellpadding="0" cellspacing="0">
    ${movies.length ? cards : emptyMsg}
  </table>
</td></tr>

<tr><td style="padding:20px 0 0;text-align:center;">
  <p style="margin:0;font-size:11px;color:#334155;font-family:monospace;">
    Movie Rating Sentinel - Automated daily digest - Ratings sourced from fee
  </p>
</td></tr>

</table>
</td></tr>
</table>
</body></html>`;
}

function buildPlainText(movies) {
  const date = new Date().toLocaleDateString("en-US", { dateStyle: "full" });
  const lines = movies.map((m, i) =>
    `${i + 1}. ${m.cleanTitle} (${m.year || "N/A"}) - IMDb ${m.imdb}/10\n` +
    (m.genre ? `Genre: ${m.genre}\n` : "") +
    (m.director ? `Dir: ${m.director}\n` : "") +
    (m.plot ? `${m.plot}\n` : "") +
    (m.imdbId ? `https://www.imdb.com/title/tt${m.imdbId}/\n` : "")
  );
  return [
    "Movie Rating Sentinel - Daily Digest",
    date,
    `Filter: IMDb >= ${CONFIG.imdbThreshold}/10`,
    "",
    `${movies.length} top-rated film(s) today:`,
    "-".repeat(50),
    ...lines,
    "-".repeat(50),
  ].join("\n");
}

```

```
// -- Step 5: Send Email -----

async function sendEmail(movies) {
  sgMail.setApiKey(CONFIG.sendgridKey);
  const subject = movies.length
    ? `${movies.length} Top-Rated Film${movies.length > 1 ? "s" : ""} Today - Mov
    : "Movie Digest - No Top-Rated Films Today";

  await sgMail.send({
    to:      CONFIG.emailTo,
    from:    CONFIG.emailFrom,
    subject,
    text:    buildPlainText(movies),
    html:    buildHTML(movies),
  });
  log.success(`Email sent to ${CONFIG.emailTo} | "${subject}"`);
}

// -- Main -----

async function main() {
  log.info("=== Movie Rating Sentinel starting ===");

  const missing = ["rssFeedUrl", "sendgridKey", "emailFrom", "emailTo"].filter((k)
  if (missing.length) {
    log.error(`Missing config: ${missing.join(", ")} - check your GitHub Secrets`
    process.exit(1);
  }

  try {
    const items      = await fetchFeed(CONFIG.rssFeedUrl);
    const topMovies = filterMovies(items);

    log.info(`${topMovies.length} / ${items.length} movies passed IMDb >= ${CONFI

    await sendEmail(topMovies);
    log.info("=== Done ===");
  } catch (err) {
    log.error(err.message);
    log.error(err.stack);
    process.exit(1);
  }
}

main();
```

